Prof. Ashwani Mishra
Assistant Professor
CSE Department

1. Integrating Angular Frontend with Express.js API

What is Integration?

Integration means connecting the frontend (Angular) with the backend (Node.js + Express) using APIs.

Angular sends **HTTP requests** to the backend server.

Example:

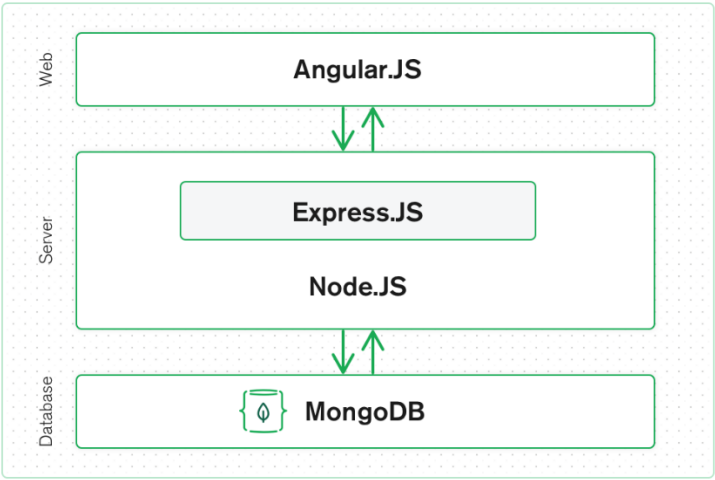
Angular → HTTP Request → Express API → MongoDB → Response → Angular

Example Application:

Student Management System:

Layer	Technology	Role
Frontend	Angular	UI
Backend	Node.js + Express	API
Database	MongoDB	Store data

Integration Architecture:



Ashwani Mishra

Flow:

User → Angular UI → HTTP Request → Express API → MongoDB → JSON Response → Angular Updates UI

Express API Example

Create backend API.

```
const express = require("express")

const app = express()

app.get("/students",(req,res)=>{

 res.json([

  {name:"Rahul"},

  {name:"Amit"},

  {name:"Riya"}

 ])

})

app.listen(3000)
```

API endpoint:

http://localhost:3000/students

Angular HTTP Service

Angular calls API using **HttpClient**.

Service example:

```
import { HttpClient } from '@angular/common/http';

constructor(private http:HttpClient){}

getStudents(){

 return this.http.get("http://localhost:3000/students");

}
```

Ashwani Mishra

Component Using Service

```
students:any;

ngOnInit(){

 this.service.getStudents().subscribe(data=>{

  this.students = data;

 });

}
```

Angular Template

```
<ul>
 <li *ngFor="let s of students">
   {{s.name}}
 </li>
</ul>
```

Output:

Rahul
Amit
Riya

2. Authentication and User Management Integration

What is Authentication?

Authentication verifies **user identity**.

Example:

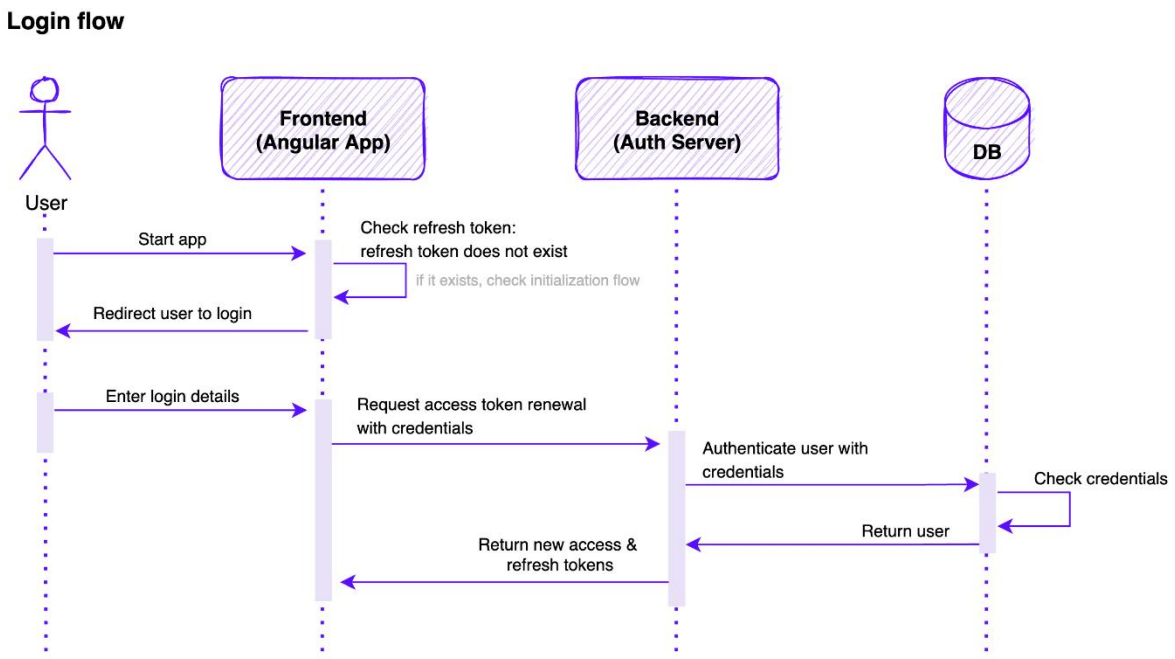
User Login

Username + Password

Server verifies credentials.

Ashwani Mishra

Authentication Flow



Flow:

User Login

↓

Angular Login Form

↓

POST /login API

↓

Express verifies credentials

↓

Generate Token

↓

Angular stores token

Express Login API Example

Install JWT:

npm install jsonwebtoken

Code:

```
const jwt = require("jsonwebtoken")

app.post("/login",(req,res)=>{

 const user = {

  id:1,

  username:"admin"

 }

 const token = jwt.sign(user,"secretkey")

 res.json({token})

})
```

Angular Login Request

```
login(data:any){
 return this.http.post("http://localhost:3000/login",data);
}
```

Token Storage

After login:

localStorage.setItem("token",response.token)

Used for **protected routes**.

User Management Features

Typical system includes:

Ashwani Mishra

Feature	Description
Register	Create account
Login	Authenticate user
Profile	View user data
Logout	End session

3. Handling Real-Time Data with WebSockets

What is WebSocket?

WebSocket allows real-time communication between client and server.

Unlike HTTP:

- HTTP = request → response
- WebSocket = continuous connection

Examples:

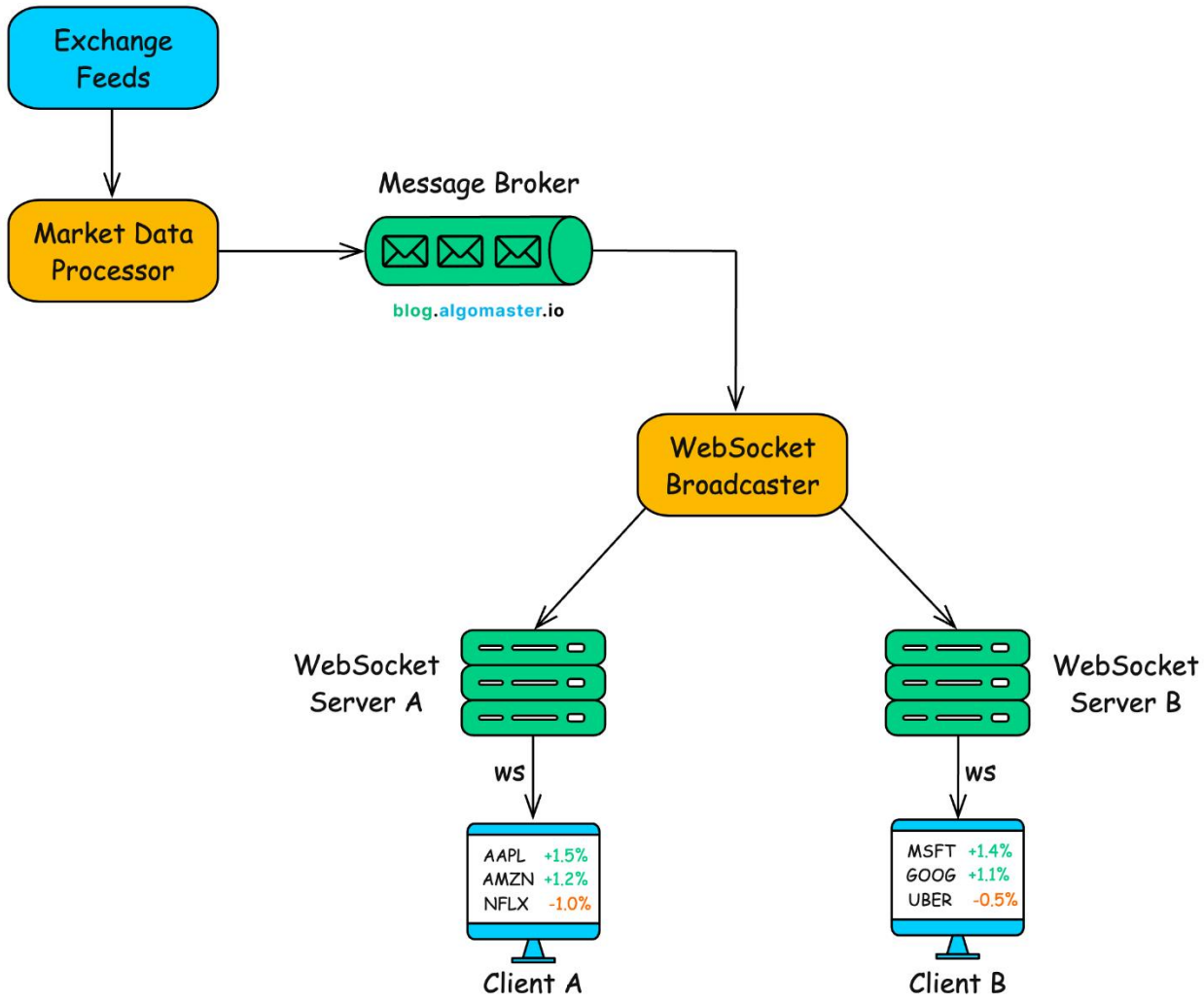
- Chat apps
- Live notifications
- Live stock price
- Online gaming

WebSocket Architecture

Flow:

Client connects → Persistent Connection → Server pushes data → Client receives instantly

Ashwani Mishra



Install Socket.io

npm install socket.io

Node.js WebSocket Server

```
const io = require("socket.io")(3001)

io.on("connection",(socket)=>{

 console.log("User connected")

 socket.emit("message","Welcome user")
```

```
})
```

Angular WebSocket Example

Install socket client:

npm install socket.io-client

Code:

```
import { io } from "socket.io-client";
const socket = io("http://localhost:3001")
socket.on("message",(data)=>{
 console.log(data)
})
```

Output:

Welcome user

Real Time Example

Live Chat Application

Users send messages instantly.

User A → Server → User B

4. Error Handling

Error handling ensures application does not crash when errors occur.

Examples:

- API failure
- Database error
- Invalid user input

Express Error Handling

```
app.use((err,req,res,next)=>{
 console.error(err)
 res.status(500).json({
  message:"Server Error"
```

Ashwani Mishra

```
})

})
```

Angular Error Handling

Example:

```
this.service.getStudents().subscribe(
 data=>{
  this.students=data
 },
 error=>{
  console.log("Error occurred")
 }
)
```

Error Handling Flow

Client Request
↓
Server Error
↓
Catch Error
↓
Send Error Response
↓
Display Message in UI

5. Testing

Testing ensures application works correctly.

Types:

Type	Description
Unit Testing	Test individual components
Integration Testing	Test API communication
End-to-End Testing	Test full application

Ashwani Mishra

Angular Testing Tools

Tool	Use
Jasmine	Testing framework
Karma	Test runner
Protractor	E2E testing

Example Unit Test

```
describe("StudentComponent",()=>{
 it("should create component",()=>{
   expect(component).toBeTruthy();
 })
})
```

Backend Testing Tool

Tool	Purpose
Mocha	Testing framework
Chai	Assertion library
Postman	API testing

Example API Test (Postman)

Test:

GET /students

Expected response:

```
[
 {"name":"Rahul"},
 {"name":"Amit"}
]
```

Complete MEAN Stack Flow:

Ashwani Mishra

Angular Frontend

↓

HTTP Request

↓

Express API

↓

MongoDB Database

↓

Response

↓

Angular UI Update

With authentication and WebSockets:

Angular

↓

JWT Authentication

↓

Express API

↓

MongoDB

↓

Real-time Updates (WebSocket)

Ashwani Mishra